MySQL 面试八股笔记

面向 Go 后端 / AI Agent 开发实习

MySQL 面试八股笔记

一、架构与 InnoDB

1. SELECT 执行流程与 MySQL 架构
2. InnoDB vs MyISAM
3. InnoDB 存储结构与 Buffer Pool

二、索引

4. B+ 树索引
5. 聚簇索引、二级索引与回表
6. 覆盖索引与索引下推
7. 联合索引与最左前缀原则
8. 索引失效场景
9. 建索引原则、唯一索引 vs 普通索引

三、事务与隔离级别

10. ACID 及实现
11. 并发问题与隔离级别
12. RR 级别与幻读

四、MVCC

13. MVCC 实现原理
14. 快照读与当前读

五、锁

15. 锁的分类
16. RR 下行锁加锁规则
17. 死锁
18. 乐观锁与悲观锁

六、日志

19. redo log
20. undo log
21. binlog 及与 redo log 的区别
22. 两阶段提交
23. UPDATE 执行全流程

七、SQL 优化

24. 慢 SQL 定位与 EXPLAIN
25. 深分页优化
26. COUNT 的几种写法

27. JOIN 与 ORDER BY 优化
八、主从复制与分库分表
28. 主从复制原理
29. GTID
30. 主从延迟
31. 分库分表
32. 分布式事务方案
九、SQL 基础与表设计
33. SQL 执行顺序
34. 常用数据类型选择
35. DELETE、TRUNCATE、DROP
36. 三大范式与反范式
37. 索引分类
38. IN 与 EXISTS、连接查询

> 使用说明：按「主题 → 知识点」组织的结构化笔记，主体参考小林coding《图解MySQL》及其面试题合集的知识框架，已重新整理改写。先记加粗关键词，再展开。涉及版本差异处已标注版本号。Redis 见 Redis 面试八股笔记。Go 秒杀综合实战见 Go 秒杀系统，其中【M-N】指向本文知识点编号。

MySQL 面试八股笔记

一、架构与 InnoDB

1. SELECT 执行流程与 MySQL 架构

- **Server 层**：
  1. **连接器**：TCP 连接、身份认证、权限获取；
  2. **查询缓存**：命中直接返回（**MySQL 8.0 已移除**，因为表一更新就失效）；
  3. **解析器**：词法分析、语法分析，生成语法树；
  4. **预处理/优化器**：检查表和字段是否存在；选择索引、决定 join 顺序，生成执行计划；
  5. **执行器**：校验权限，调用存储引擎接口逐行读取并判断条件。
- **存储引擎层**：负责数据存取，插件式（InnoDB、MyISAM 等）。

延伸 / 易错点

- **长连接问题**：连接占用内存累积，可定期断开或执行 `mysql_reset_connection`。

2. InnoDB vs MyISAM

维度	InnoDB	MyISAM
事务	支持	不支持
锁粒度	**行级锁**	表级锁
外键	支持	不支持
崩溃恢复	支持（redo log）	不支持
索引	**聚簇索引**（数据即索引）	非聚簇，索引存数据地址
MVCC	支持	不支持
`COUNT(*)`	需扫描	保存总行数（无 WHERE 时很快）

- MySQL 5.5 起默认引擎为 InnoDB。

3. InnoDB 存储结构与 Buffer Pool

- 逻辑结构：**表空间 → 段 → 区（1MB，64 个页）→ 页（默认 16KB）→ 行**。页是磁盘与内存交互的**最小单位**。
- 行格式：COMPACT、**DYNAMIC（5.7 起默认）**、COMPRESSED；行记录含变长字段长度列表、NULL 值列表、记录头、隐藏列。
- **Buffer Pool**：缓存数据页和索引页，读写先操作内存，脏页由后台异步刷盘。
  - 改进 LRU：分 **young 区和 old 区**（old 默认占 37%，`innodb_old_blocks_pct`），新页先插入 old 区头部，在 old 区停留超过 `innodb_old_blocks_time`（默认 1000ms）后再次访问才进 young 区 —— 解决**预读失效**和**全表扫描导致的缓存污染**。
  - 包含 free 链表、flush 链表（脏页）。
- 其他：**Change Buffer**（缓存二级非唯一索引的修改）、**Doublewrite Buffer**（防止页部分写入）、**自适应哈希索引**。

延伸 / 易错点

- **双写缓冲的作用**：16KB 页写到一半宕机（部分写），redo log 无法基于损坏页恢复，先写双写区保证有完整副本。

二、索引

4. B+ 树索引

- 目标：**减少磁盘 IO**，树越矮越好。
- 对比：
  - **B 树**：非叶子节点也存数据，单页能存的键少，树更高；范围查询需中序遍历。
  - **B+ 树**：**非叶子节点只存键和指针，扇出大、树矮**；**所有数据在叶子节点**，叶子之间**双向链表**相连，范围查询高效；查询稳定（都到叶子层）。

    - **哈希**：等值 O(1)，但**不支持范围、排序、最左前缀**。
    - **二叉/红黑树**：树高太高，IO 次数多。
    - **跳表**：层高随数据增长较高，IO 次数多于 B+ 树。
- 估算：主键 BIGINT（8B）+ 页指针（6B）≈ 14B，一个 16KB 非叶子页约存 **1170** 个指针；若一行约 1KB，叶子页存 16 行，**3 层约 1170 × 1170 × 16 ≈ 2000 万行**，根节点常驻内存时只需 1~3 次 IO。

延伸 / 易错点

- **「单表 2000 万」不是硬性规定**：只是按上述行大小估算的经验值，行越大阈值越小。

5. 聚簇索引、二级索引与回表

- **聚簇索引（主键索引）**：叶子节点存**完整行数据**，一张表只有一个。若无主键，选第一个非空唯一索引；都没有则使用隐藏的 6 字节 `DB_ROW_ID`。
- **二级索引（辅助索引）**：叶子节点存**索引列 + 主键值**。
- **回表**：通过二级索引查到主键后，再到聚簇索引中查完整行。

延伸 / 易错点

- **推荐自增主键**：顺序追加写入，**避免页分裂**和随机 IO；主键短，二级索引更省空间。UUID 无序且长，插入易页分裂。自增主键不保证连续（事务回滚、唯一键冲突、批量插入预分配都会产生空洞）。
- **二级索引存主键而非行地址**：行移动（页分裂）时无需更新二级索引。

6. 覆盖索引与索引下推

- **覆盖索引**：查询的列**全部在索引中**，无需回表。EXPLAIN 的 Extra 显示 `Using index`。优化手段：为高频查询建立联合索引覆盖所需列。
- **索引下推 ICP（5.6 引入）**：联合索引遍历时，**在存储引擎层**就用索引中包含的列过滤不满足条件的记录，减少回表次数。Extra 显示 `Using index condition`。
    - 例：索引 (name, age)，`WHERE name LIKE '张%' AND age = 10`，无 ICP 时每条 name 匹配都回表；有 ICP 时先在索引里判断 age。

7. 联合索引与最左前缀原则

- 联合索引 `(a, b, c)` 按 a 排序，a 相同按 b，b 相同按 c，因此**必须从最左列开始连续匹配**才能利用索引定位。
- 可用：`a`、`a,b`、`a,b,c`；`WHERE b=1 AND a=1` 也可（优化器会调整顺序）。
- 不可用（定位）：`b`、`c`、`b,c`。
- `a=1 AND c=3`：只有 a 用于定位，c 可借助 ICP 在索引内过滤。
- **范围查询**：`a > 1 AND b = 2`，a 之后的 b 无法用于定位（a 范围内 b 无序）；但 `>=`、`BETWEEN`、`LIKE 'x%'` 在边界值上 b 仍可部分利用。

- 8.0.13 引入 **索引跳跃扫描（Skip Scan）**，某些条件下缺少最左列也能利用联合索引。

延伸 / 易错点

- **列顺序**：区分度高的列、等值条件列放前面；兼顾覆盖索引与排序需求。

8. 索引失效场景

- 对索引列做**函数或表达式运算**：`WHERE YEAR(create_time)=2024`、`WHERE id+1=10`。
- **隐式类型转换**：字符串列用数字比较 `WHERE phone = 13800000000`（MySQL 将字符串转数字，相当于对列用函数）。反之，整数列和字符串比较不影响索引。
- **左模糊/全模糊**：`LIKE '%abc'`（覆盖索引情况下可能走索引全扫描）。
- **不满足最左前缀**。
- **OR** 条件中有一侧列无索引。
- 优化器根据成本判断**全表扫描更快**（如回表比例很高）。
- `!=`、`NOT IN`、`IS NOT NULL` 等可能因成本原因不走索引（并非一定失效）。

延伸 / 易错点

- **确认是否走索引**：`EXPLAIN` 看 `key`、`type`、`rows`；必要时 `FORCE INDEX` 干预。

9. 建索引原则、唯一索引 vs 普通索引

- 适合：WHERE、JOIN、ORDER BY、GROUP BY 频繁的列；**区分度高**的列；考虑联合索引与覆盖索引。
- 不适合：区分度低（如性别）、频繁更新、数据量很小的表；索引不宜过多（影响写入、占空间）。
- 长字符串用**前缀索引**（`INDEX(email(10))`），但无法用于覆盖索引和排序。
- **唯一 vs 普通**：查询性能几乎无差别；更新时**普通索引可使用 Change Buffer**（不需立即读入页），唯一索引必须读页判断唯一性。业务能保证唯一时，写多读少场景可优先普通索引。

三、事务与隔离级别

10. ACID 及实现

- **原子性 A**：要么全成功要么全回滚 → **undo log**。
- **一致性 C**：数据满足完整性约束，是目的 → 由 A、I、D 共同保证 + 应用层保证。
- **隔离性 I**：并发事务互不干扰 → **锁 + MVCC**。
- **持久性 D**：提交后永久保存 → **redo log**（WAL）。

11. 并发问题与隔离级别

- **脏读**：读到其他事务**未提交**的数据。

- **不可重复读**：同一事务内两次读**同一行**，值不同（被其他事务修改并提交）。
- **幻读**：同一事务内两次**范围查询**，记录**条数**不同（被其他事务插入/删除）。

隔离级别	脏读	不可重复读	幻读
读未提交 RU	可能	可能	可能
读已提交 RC	避免	可能	可能
可重复读 RR（**InnoDB 默认**）	避免	避免	可能（InnoDB 很大程度避免）
串行化	避免	避免	避免

- 实现：RC、RR 基于 **MVCC 的 ReadView**；串行化对读加共享锁。

延伸 / 易错点

- **很多公司用 RC 的原因**：RC 没有间隙锁，**锁冲突和死锁更少**、并发更高；早期 binlog statement 格式下 RC 有主从不一致问题，改用 ROW 格式后可用。

12. RR 级别与幻读

- **快照读**（普通 SELECT）：通过 MVCC，事务内读同一 ReadView，**看不到**其他事务新插入的数据。
- **当前读**（`SELECT ... FOR UPDATE`、UPDATE、DELETE）：通过 **Next-Key Lock（记录锁 + 间隙锁）** 阻止其他事务在范围内插入。
- **没有完全解决**，反例：事务 A 快照读未查到 id=5 → 事务 B 插入 id=5 并提交 → 事务 A `UPDATE ... WHERE id=5` 成功（当前读），该行 trx_id 变为 A 自己 → A 再快照读即可看到 id=5，出现幻读。另外先快照读后当前读也可能读到不同结果。
- 规避：需要时一开始就用 `FOR UPDATE` 加锁。

四、MVCC

13. MVCC 实现原理

- 三要素：**隐藏列 + undo log 版本链 + ReadView**。
- **隐藏列**：`DB_TRX_ID`（最近修改该行的事务 ID）、`DB_ROLL_PTR`（回滚指针，指向 undo log 中旧版本）、`DB_ROW_ID`（无主键时）。
- **版本链**：每次修改把旧版本写入 undo log，通过 roll_ptr 串成链表。
- **ReadView** 四个字段：
  - `m_ids`：创建时**活跃（未提交）事务 ID 列表**；
  - `min_trx_id`：m_ids 中最小值；
  - `max_trx_id`：创建时系统**下一个将分配的事务 ID**（不是 m_ids 最大值）；
  - `creator_trx_id`：创建该 ReadView 的事务 ID。

- **可见性判断**（对版本的 trx_id）：
  1. `trx_id == creator_trx_id` → 自己改的，可见；
  2. `trx_id < min_trx_id` → 已提交，可见；
  3. `trx_id >= max_trx_id` → 在 ReadView 之后开启，不可见；
  4. 介于之间：在 m_ids 中 → 未提交，不可见；不在 → 已提交，可见。
  - 不可见则沿版本链找上一个版本继续判断。
- **RC vs RR**：RC **每次 SELECT 都生成新 ReadView**；RR **只在事务中第一次 SELECT 时生成**，之后复用。

延伸 / 易错点

- **只读事务**：不分配真正的事务 ID，首次修改数据时才分配。
- **undo 清理**：没有任何 ReadView 需要该版本时，由 **purge 线程**清理。

14. 快照读与当前读

- **快照读**：普通 `SELECT`，读 MVCC 历史版本，不加锁。
- **当前读**：读**最新已提交版本**并加锁：`SELECT ... FOR UPDATE`（排他锁）、`SELECT ... LOCK IN SHARE MODE` / 8.0 `FOR SHARE`（共享锁）、`INSERT`、`UPDATE`、`DELETE`。

五、锁

15. 锁的分类

- **全局锁**：`FLUSH TABLES WITH READ LOCK`，全库只读，用于全库逻辑备份（InnoDB 更推荐 `mysqldump --single-transaction`，基于 MVCC 不加全局锁）。
- **表级锁**：
  - 表锁：`LOCK TABLES ... READ/WRITE`；
  - **元数据锁 MDL**：增删改查加 MDL 读锁，改表结构加 MDL 写锁；长事务 + DDL 可能导致后续请求全部阻塞；
  - **意向锁 IS/IX**：加行锁前先在表上加意向锁，使加表锁时**无需逐行检查**；意向锁之间互相兼容；
  - **AUTO-INC 锁**：自增 ID 分配（`innodb_autoinc_lock_mode`，8.0 默认 2 为轻量交错模式）。
- **行级锁（InnoDB）**：
  - **Record Lock 记录锁**：锁单条索引记录；
  - **Gap Lock 间隙锁**：锁记录间的间隙（开区间），**只为防止插入**，间隙锁之间**互相兼容**；
  - **Next-Key Lock**：记录锁 + 间隙锁，**左开右闭区间**，RR 下加锁的基本单位；
  - **插入意向锁**：INSERT 前加的特殊间隙锁，与间隙锁冲突。

- 行锁是**加在索引上的**；没有走索引的更新会扫描全表，对每条记录加 next-key lock，相当于锁全表。

延伸 / 易错点

- **S/X 兼容性**：S 与 S 兼容，S 与 X、X 与 X 互斥。

16. RR 下行锁加锁规则

- 原则 1：加锁基本单位是 **next-key lock（前开后闭）**。
- 原则 2：查找过程中**访问到的对象才加锁**。
- 优化 1：**唯一索引等值查询且记录存在**，next-key lock **退化为记录锁**。
- 优化 2：索引上等值查询，向右遍历到**最后一个不满足条件的值**时，next-key lock **退化为间隙锁**。
- 例：唯一索引 id 有 5、10、15，`WHERE id = 7 FOR UPDATE` → 锁间隙 (5, 10)。
- 覆盖索引下的 `LOCK IN SHARE MODE` 只锁二级索引，不锁主键索引；`FOR UPDATE` 会同时锁主键索引上对应记录。

延伸 / 易错点

- **查看锁**：8.0 用 `performance_schema.data_locks`、`data_lock_waits`。

17. 死锁

- 四个必要条件：**互斥、占有并等待、不可剥夺、循环等待**。
- 常见场景：两个事务以**不同顺序**更新相同行；RR 下两个事务先后对同一间隙加间隙锁（互相兼容），再各自插入，互相等待插入意向锁。
- InnoDB 处理：
  - **死锁检测**（`innodb_deadlock_detect` 默认开启）：等待图检测环，**回滚代价较小（undo 量少）的事务**；
  - **锁等待超时** `innodb_lock_wait_timeout` 默认 **50 秒**。
- 排查：`SHOW ENGINE INNODB STATUS` 查看 `LATEST DETECTED DEADLOCK`。
- 避免：**固定加锁顺序**、**缩短事务**、合理索引减少锁范围、降低隔离级别到 RC、大事务拆分。

18. 乐观锁与悲观锁

- **悲观锁**：假设冲突频繁，操作前加锁，如 `SELECT ... FOR UPDATE`。适合**写多、冲突多**。
- **乐观锁**：假设冲突少，提交时校验，常用**版本号/CAS**：`UPDATE t SET stock=stock-1, version=version+1 WHERE id=1 AND version=#{v}`，影响行数为 0 则重试。适合**读多写少**。
- 库存扣减常用：`UPDATE t SET stock=stock-1 WHERE id=1 AND stock>0`。

六、日志

19. redo log

- InnoDB 引擎层的**物理日志**，记录「某个数据页的某处做了什么修改」。
- **WAL（Write-Ahead Logging）**：修改先写 redo log 再异步刷脏页；redo log 是**顺序写**，比随机写数据页快得多。
- 作用：**崩溃恢复（crash-safe）**，保证持久性。
- 结构：传统为固定大小的一组文件**循环写**，`write pos` 为写入位置，`checkpoint` 为已刷盘可擦除位置，追上时需先刷脏页推进 checkpoint（8.0.30 起由 `innodb_redo_log_capacity` 统一配置）。
- 刷盘策略 `innodb_flush_log_at_trx_commit`：
  - **1（默认）**：每次提交都写入并 fsync，最安全；
  - 0：每秒写入并刷盘，宕机可能丢 1 秒；
  - 2：每次提交写入 OS page cache，每秒 fsync，MySQL 崩溃不丢、操作系统崩溃可能丢。

20. undo log

- **逻辑日志**，记录修改前的数据（insert 记主键用于删除；delete/update 记旧值）。
- 作用：**事务回滚（原子性）** + **MVCC 版本链**。
- 已提交事务的 undo 在不再被 ReadView 需要后由 purge 线程清理；**长事务会导致 undo 无法清理、空间膨胀**。
- undo log 本身的修改也会记录 redo log。

21. binlog 及与 redo log 的区别

- binlog 是 **Server 层**的**逻辑日志**，记录所有修改数据和表结构的操作，用于**主从复制和数据恢复**。
- 格式：
  - `STATEMENT`：记录 SQL，体积小，但 `NOW()`、`UUID()` 等可能导致主从不一致；
  - `ROW`（**5.7.7 起默认**）：记录行变化前后的值，准确但体积大；
  - `MIXED`：自动选择。
- 刷盘 `sync_binlog`：0 交给 OS；**1 每次提交 fsync（8.0 默认）**；N 每 N 个事务 fsync。

对比	redo log	binlog
层级	InnoDB 引擎层	Server 层，所有引擎可用
类型	物理日志（页修改）	逻辑日志（SQL 或行变化）
写入	循环写，空间固定	**追加写**，写满换新文件
用途	崩溃恢复	主从复制、按时间点恢复

延伸 / 易错点

- **为什么两种日志都需要**：binlog 不记录数据页刷盘状态，无法用于崩溃恢复；redo log 循环写会覆盖，无法做历史归档和主从复制。

22. 两阶段提交

- 目的：保证 **redo log 和 binlog 的逻辑一致**，避免主从数据不一致或恢复后数据不一致。
- 流程（内部 XA 事务）：
  1. **prepare 阶段**：写 redo log，状态标记为 prepare，刷盘；
  2. **写 binlog** 并刷盘；
  3. **commit 阶段**：redo log 标记 commit。
- 崩溃恢复判断：
  - redo 已 commit → 提交；
  - redo 为 prepare：**binlog 完整（能找到对应 XID）→ 提交；否则 → 回滚**。
- 问题：每次提交两次 fsync 性能差 → **组提交（group commit）**，多个事务合并一次刷盘。

23. UPDATE 执行全流程

1. 执行器调用引擎取目标行：所在页在 Buffer Pool 中则直接返回，否则从磁盘读入（加当前读锁）；
2. 记录 **undo log**（旧值），用于回滚和 MVCC；
3. 更新 Buffer Pool 中的数据页，标记为脏页；
4. 写 **redo log**（prepare）；
5. 写 **binlog**；
6. 提交，redo log 置为 commit；
7. 后台线程择机将脏页刷盘。

七、SQL 优化

24. 慢 SQL 定位与 EXPLAIN

- 定位：开启**慢查询日志** `slow_query_log=ON`，`long_query_time`（默认 10 秒，一般调小到 1 秒或更低）；`mysqldumpslow`/`pt-query-digest` 分析；`SHOW PROCESSLIST` 看当前执行。
- 分析：**EXPLAIN** 关键字段：
  - `type`（由好到差）：`system > const > eq_ref > ref > range > index > ALL`，至少达到 range；
  - `key`：实际使用的索引；`key_len`：使用的索引长度（判断联合索引用到几列）；
  - `rows`：预估扫描行数；

- `Extra`：`Using index`（覆盖索引）、`Using index condition`（ICP）、`Using where`、**`Using filesort`（额外排序）**、**`Using temporary`（临时表）** —— 后两者需重点优化。
- 优化手段：加/调整索引、避免 `SELECT *`、避免索引失效写法、改写子查询为 JOIN、大事务拆分、读写分离、缓存。

25. 深分页优化

- 原因：MySQL 需扫描并**丢弃前 100 万行**（且每行可能回表）。
- 方案：
  - **延迟关联（子查询先查主键）**：`SELECT t.* FROM t JOIN (SELECT id FROM t ORDER BY id LIMIT 1000000, 10) x ON t.id = x.id`，子查询走覆盖索引；
  - **游标/书签分页**：记录上一页最后的 id，`WHERE id > last_id ORDER BY id LIMIT 10`（适合「下一页」场景，不支持随机跳页）；
  - 业务限制最大页数，或使用 ES 等搜索引擎。

26. COUNT 的几种写法

- `COUNT(*)` 与 `COUNT(1)`：统计**所有行**，InnoDB 中处理方式相同、性能一致，官方推荐 `COUNT(*)`。
- `COUNT(主键)`：需读取主键值判断非空。
- `COUNT(普通列)`：**不统计 NULL 值**，需读取该列。
- 性能大致：`COUNT(*) ≈ COUNT(1) > COUNT(主键) > COUNT(列)`。优化器会选择**最小的二级索引**遍历来计数。
- InnoDB 因 MVCC 不同事务可见行数不同，**不能像 MyISAM 那样直接存总行数**；大表计数可用 Redis 计数器或单独计数表。

27. JOIN 与 ORDER BY 优化

- JOIN 算法：
  - **Index Nested-Loop Join**：被驱动表关联字段有索引，逐行查索引，效率高；
  - **Block Nested-Loop Join**：被驱动表无索引，将驱动表数据放入 `join_buffer` 批量比较；**8.0.18 起引入 Hash Join**，8.0.20 起取代 BNL。
- 原则：**小表驱动大表**（「小」指过滤后参与 join 的数据量）；**被驱动表关联字段加索引**；避免过多表 JOIN。
- ORDER BY：
  - 利用索引有序性避免 `filesort`（排序字段与 WHERE 等值字段构成联合索引）；
  - filesort 使用 `sort_buffer`，数据量大时用临时文件归并排序；
  - 区分全字段排序和 rowid 排序（`max_length_for_sort_data`）。

八、主从复制与分库分表

28. 主从复制原理

- 三个线程：
  1. 主库 **binlog dump 线程**：将 binlog 发送给从库；
  2. 从库 **IO 线程**：接收 binlog 写入**中继日志 relay log**；
  3. 从库 **SQL 线程**：读取 relay log **回放**。
- 复制模式：
  - **异步复制**（默认）：主库提交不等从库，可能丢数据；
  - **半同步复制**：至少一个从库**接收并写入 relay log** 后返回 ACK 才响应客户端；5.7 引入 `AFTER_SYNC`（**无损半同步，默认**）：在写 binlog 后、引擎提交前等待 ACK；
  - **组复制 MGR**：基于 Paxos 类协议多数派确认。
- 用途：读写分离、高可用、备份。

29. GTID

- **GTID（全局事务 ID）** = `server_uuid:transaction_id`，每个事务在整个复制集群中唯一。
- 作用：从库通过 `MASTER_AUTO_POSITION=1` **自动定位复制位点**，无需手动指定 binlog 文件名和偏移量；**故障切换**时新主、从库可依据已执行的 GTID 集合自动对齐；天然避免同一事务被重复执行。

30. 主从延迟

- 原因：从库机器性能差、从库查询压力大、**大事务**（回放耗时长）、网络延迟、早期**单线程回放**。
- 并行复制演进：**5.6 按库并行**；**5.7 基于组提交的 LOGICAL_CLOCK 并行**；**8.0 基于 WRITESET 并行**（无冲突事务可并行）。
- 业务应对：
  - 写后立即读的场景**强制读主库**；
  - 判断从库位点/GTID 追上后再读；
  - 关键数据走缓存；
  - 拆分大事务。

31. 分库分表

- 时机：单表数据量过大（常见经验值单表千万级、容量数十 GB）导致查询和 DDL 变慢 → **分表**；单库连接数、QPS、磁盘达到瓶颈 → **分库**。优先考虑索引优化、缓存、读写分离、归档。
- 方式：

    - **垂直分库**：按业务拆分（用户库、订单库）；
    - **垂直分表**：大字段/冷字段拆到扩展表；
    - **水平分表/分库**：按分片键将行分散到多表/多库。
- 分片策略：**Hash 取模**（分布均匀，扩容需迁移）、**Range 范围**（按时间/ID，易扩容但有热点）、一致性哈希、**基因法**（订单 ID 嵌入用户 ID 部分位，便于按两种维度查询）。
- 带来的问题：
    - **全局唯一 ID**：雪花算法（1 位符号 + 41 位毫秒时间戳 + 10 位机器 ID + 12 位序列号，存在**时钟回拨**问题）、号段模式（美团 Leaf）、Redis INCR；
    - **跨库 JOIN**：冗余字段、应用层组装、宽表/ES；
    - **分布式事务**：2PC/XA、TCC、本地消息表、Seata AT、最大努力通知；
    - **跨分片分页、排序、聚合**：归并计算或借助 ES；
    - **扩容迁移**：成倍扩容、双写迁移。
- 中间件：**ShardingSphere（ShardingSphere-JDBC/Proxy）**、MyCat、Vitess（Go 实现，YouTube 开源）。

延伸 / 易错点

- **分片键选择**：选**查询最频繁、分布均匀**的字段（如订单表用 user_id），尽量让查询落在单分片。

32. 分布式事务方案

- **2PC / XA**：协调者 prepare → commit；强一致，但同步阻塞、协调者单点、性能差。
- **TCC**：Try（预留资源）、Confirm（确认）、Cancel（取消）；业务侵入大，需处理**空回滚、幂等、悬挂**。
- **本地消息表**：业务操作和消息写同一本地事务，后台扫描投递 MQ，消费方幂等，最终一致。
- **事务消息**（RocketMQ 半消息）：先发半消息 → 执行本地事务 → 提交/回滚消息，回查机制。
- **Seata AT**：自动生成反向 SQL（undo_log）回滚，基于两阶段与全局锁。
- **Saga**：长事务拆为多个本地事务，失败则执行补偿。

九、SQL 基础与表设计

33. SQL 执行顺序

- 书写顺序：`SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT`。
- **逻辑执行顺序**：`FROM → ON → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT`。
- 推论：WHERE 中**不能使用 SELECT 中定义的别名**和聚合函数（聚合条件放 HAVING）；ORDER BY 可以使用别名（MySQL 在 GROUP BY/HAVING 中也做了别名扩展支持）。

34. 常用数据类型选择

- **CHAR(n) vs VARCHAR(n)**：CHAR 定长（最多 255 字符），不足补空格，适合长度固定的值（如 MD5）；VARCHAR 变长，额外 1~2 字节记录长度。MySQL 5.0+ 中 **n 表示字符数**，单行所有列总长度上限 65535 字节。
- **INT(11) 中的 11**：只是**显示宽度**（配合 ZEROFILL），不影响存储范围；8.0.17 起整数显示宽度已废弃。
- **金额用 DECIMAL**：定点数精确存储，FLOAT/DOUBLE 是二进制浮点数会有精度误差；也可用 BIGINT 存「分」。
- **IP 地址**：可用 `INT UNSIGNED` 配合 `INET_ATON()` / `INET_NTOA()` 存 IPv4，节省空间且便于范围查询；IPv6 可用 `VARBINARY(16)` + `INET6_ATON()`。
- **TEXT 不是无限大**：TEXT 最大 64KB，MEDIUMTEXT 16MB，LONGTEXT 4GB；大字段建议拆表或存对象存储。
- **NULL vs 空字符串**：NULL 表示未知，不占数据空间但需 NULL 值列表标记；`COUNT(列)` 不统计 NULL，`= NULL` 恒不成立需用 `IS NULL`；建议列设 `NOT NULL DEFAULT ''`。

35. DELETE、TRUNCATE、DROP

对比	DELETE	TRUNCATE	DROP
类型	DML	DDL	DDL
作用	删除行（可带 WHERE）	清空全表数据，保留表结构	删除表结构和数据
回滚	可回滚（写 undo）	不可回滚	不可回滚
自增 ID	不重置	重置	—
速度	逐行删除，慢	快（重建表）	快
触发器	触发	不触发	不触发

36. 三大范式与反范式

- **第一范式**：每列都是不可再分的原子值。
- **第二范式**：在 1NF 基础上，非主键列**完全依赖于整个主键**（消除对联合主键的部分依赖）。
- **第三范式**：在 2NF 基础上，非主键列**不传递依赖**于主键（消除非主键列之间的依赖）。
- **反范式**：互联网业务常为减少 JOIN、提升查询性能而**适当冗余字段**，以空间换时间，代价是需要维护数据一致性。

37. 索引分类

- **按数据结构**：B+ 树索引（InnoDB 默认）、哈希索引（Memory 引擎支持；InnoDB 有自适应哈希索引，自动构建）、全文索引（FULLTEXT）。
- **按物理存储**：聚簇索引（主键索引）、二级索引（辅助索引）。

- **按字段特性**：主键索引（唯一且非空，一张表一个）、唯一索引（值唯一，允许多个 NULL）、普通索引、前缀索引。
- **按字段个数**：单列索引、联合索引。
- 索引优缺点：加快查询、排序和分组；但**占用空间**、**降低写入速度**（增删改需维护 B+ 树，可能页分裂）。

38. IN 与 EXISTS、连接查询

- **IN**：先执行子查询得到结果集，再与外表匹配，适合**子查询结果集小**；**EXISTS**：遍历外表，逐行判断子查询是否有结果，适合**外表小、子查询表大且有索引**。新版本优化器会将两者改写为半连接（semi-join），差异变小。
- **NOT IN 陷阱**：子查询结果含 NULL 时 `NOT IN` 返回空集，建议用 `NOT EXISTS` 或 `LEFT JOIN ... IS NULL`。
- **连接类型**：`INNER JOIN` 取交集；`LEFT JOIN` 保留左表全部行，右表无匹配补 NULL；`RIGHT JOIN` 相反；MySQL 不支持 `FULL JOIN`，可用 `UNION` 模拟。
- **避免重复插入**：唯一索引 + `INSERT IGNORE` / `INSERT ... ON DUPLICATE KEY UPDATE` / `REPLACE INTO`（先删后插）。